

java-fundamentals-course-chronisteam

Resumen del Curso

Este curso introductorio de 1 hora enseña a estudiantes de secundaria los conceptos básicos de Programación orientada a objetos (POO), utilizando exclusivamente herramientas en línea como Replit y JDoodle. No requiere experiencia previa en programación ni instalación de software adicional, solo un navegador web.

Los estudiantes aprenderán qué son las clases y los objetos, cómo definir atributos (datos) y métodos (acciones), y cómo modelar situaciones de la vida real usando POO. Al final crearán un pequeño programa orientado a objetos.

Duración total: 60 minutos

Público objetivo: Estudiantes de 12 a 17 años sin experiencia previa en programación

Prerrequisitos: Ninguno

Herramientas necesarias: Navegador web

Repositorio de código fuente: <https://github.com/java-fundamentals-course-chronisteam/java-fundamentals-course-chronisteam>

Nota: Este archivo **README** fue actualizado para registrar la configuración final del curso.

Link de la app para poder practicar tu código:

link: <https://java-fundamentals-course-program-ch.vercel.app/>

Secuencia de la Lección

Lección 1: Introducción a Java y la programación, sintaxis básica: variables, tipos de datos. (5 minutos)

Responsable: Andreow Jomark Santiago Peña

- **Descripción:** Breve introducción sobre qué es programar, presentación del instructor y objetivo: entender variables y tipos básicos (int, double, String, boolean). Incluye demostración con el programa **HolaMundo** en Replit.
- **Enlace del video:** <https://youtu.be/prrAjmJ7wLY>
- **Editor online:** <https://replit.com/join/ewucjjgzne-andreows>
- **Introducción:**

¡Hola a todos! Bienvenidos a la Lección 1 del curso de Fundamentos de Java para Principiantes. Mi nombre es Andreow Santiago. En los próximos minutos daremos nuestros primeros pasos en programación con Java.

¿Alguna vez te preguntaste cómo le dices a una computadora exactamente qué hacer? Eso es programar. Java es un lenguaje muy usado para crear aplicaciones; en esta lección veremos los conceptos básicos que necesitas para empezar.

Hoy aprenderemos qué es una variable y los tipos de datos más comunes en Java. Es una lección corta y práctica, así que ¡vamos a comenzar!

- **Conclusiones clave:**

- Java se usa para aplicaciones y juegos.
- Las variables son como cajitas que guardan datos.
- Los tipos de datos definen qué tipo de información puede guardar una variable.

- **Código ejemplo:** `HolaMundo.java`

```
public class HolaMundo {
    public static void main(String[] args) {
        // Creamos nuestras variables
        int edad = 16;
        double altura = 1.70;
        String nombre = "Ana";
        boolean esEstudiante = true;

        // Mostramos el contenido de las variables
        System.out.println("Hola, me llamo " + nombre);
        System.out.println("Tengo " + edad + " años.");
        System.out.println("Mido " + altura + " metros.");
        System.out.println("¿Soy estudiante? " + esEstudiante);
    }
}
```

- **Práctica:** [Clic para programar](#)

Lección 2: Operadores y estructuras de control, métodos, funciones y parámetros. (12:40 minutos)

Responsable: Carlos Alberto Lopez Goitia

- **Descripción:** En esta lección practicamos operadores aritméticos (+ - * / %), comparación (>, ==), uso de booleanos, y estructuras de control: `if/else`, `else if`, `for`, `while`. Además, creamos y usamos métodos con y sin retorno: `saludar(String)` y `sumar(int,int)`.
- **Enlace del video:** <https://youtu.be/RryyYpLgSpk>
- **Editor online:** [Replit - Operadores y estructuras de control, métodos, funciones y parámetros](#)
- **Consejos clave:**
 - Con enteros, `a / b` hace división entera ($10/3 \rightarrow 3$). Para decimales: `(double)a / b`.
 - `%` devuelve el resto (útil para saber si un número es par: `n % 2 == 0`).
 - `if / else if / else` se evalúan en orden; la primera verdadera corta el flujo.
 - En `for (int i = 1; i <= 10; i++)` cuida los límites.
 - En `while` recuerda actualizar la variable de control.

- Un método **void** no retorna; si tiene tipo de retorno, debe usar **return**.

- **Código inicial utilizado en la exposición:** Main.java

```
public class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = 3;

        int suma = a + b;
        int resta = a - b;
        int multiplicacion = a * b;
        int division = a / b;        // división entera
        int resto = a % b;          // módulo (resto)

        System.out.println("Suma: " + suma);
        System.out.println("Resta: " + resta);
        System.out.println("Multiplicación: " + multiplicacion);
        System.out.println("División: " + division);
        System.out.println("Resto: " + resto);

        boolean esMayor = a > b;
        boolean esIgual = a == b;

        System.out.println("¿a es mayor que b? " + esMayor);
        System.out.println("¿a es igual a b? " + esIgual);

        System.out.println("-----");

        int nota = 15;
        if (nota >= 11){
            System.out.println("Aprobaste");
        } else {
            System.out.println("Desaprobaste");
        }

        System.out.println("-----");

        int edad = 16;
        if (edad >= 18){
            System.out.println("Eres mayor de edad");
        } else if (edad >= 13){
            System.out.println("Eres adolescente");
        } else {
            System.out.println("Eres niño");
        }

        System.out.println("-----");

        for (int i = 1; i <= 10; i++){
            System.out.println("Vuelta numero: " + i);
        }
    }
}
```

```
System.out.println("-----");

int contador = 1;
while (contador <= 3){
    System.out.println("Intento: " + contador);
    contador = contador + 1;
}

System.out.println("-----");

saludar("Snay");
saludar("Alberto");

int resultado = sumar(5, 7);
System.out.println("La suma es: " + resultado);

}

static void saludar(String nombre){
    System.out.println("Hola " + nombre + ", bienvenido al curso de Java");
}

static int sumar(int a, int b){
    int respuesta = a + b;
    return respuesta;
}

}
```

- **Práctica:** [Clic para programar](#)

1. Cambia **a** y **b** y prueba **(double)a / b** para ver decimales.
2. Crea **static int multiplicar(int x, int y)** y úsala en **main**.
3. Escribe **static boolean esPar(int n)** y muestra "par"/"impar".
4. Con un **for**, imprime los números del 10 al 1.
5. Pide otra **nota** y prueba todos los caminos del **if**.

Lección 3: Entrada/salida con Scanner, Arreglos simples o cadenas. (18:44 minutos)

Responsable:

- **Descripción:** En este módulo presentamos cómo interactuar con el usuario en Java usando la consola. Primero recordamos la salida básica con `System.out.println`, luego introducimos la clase `Scanner` para leer datos del teclado, trabajando tanto con números (`nextInt`) como con texto (`nextLine`). Después, explicamos la idea de arreglos simples (`int[]` y `String[]`) con ejemplos de notas y materias favoritas, y finalmente combinamos `Scanner` + arreglos para guardar y mostrar los nombres de tres amigos. Todo se desarrolla con ejemplos cortos y sin bucles, pensado para estudiantes que recién comienzan a programar.

- **Enlace del video:** <https://youtu.be/aToAE5dtBlw>
- **Editor online:** <https://replit.com/@smbmontalvo/JavaCourse>
- **Consejos clave:**

Recuerda importar Scanner: Siempre que uses Scanner, la primera línea debe ser `import java.util.Scanner;`

Crea un solo Scanner por programa: Usa algo como `Scanner teclado = new Scanner(System.in);` al inicio del main y ciérralo al final con `teclado.close();`

Diferencia entre `nextInt()` y `nextLine()`:

`nextInt()` → lee números enteros.

`nextLine()` → lee una línea completa de texto. Ten claro qué tipo de dato quieres leer antes de elegir el método.

Los arreglos empiezan en 0: El primer elemento está en la posición 0, no en la 1. En un arreglo de tamaño 3, los índices válidos son 0, 1 y 2.

Un arreglo solo guarda un tipo de dato: `int[]` solo números enteros, `String[]` solo cadenas de texto. No se pueden mezclar tipos distintos dentro del mismo arreglo.

Piensa en el arreglo como "varios casilleros": Imagina que cada posición del arreglo es un casillero donde guardas un valor. Eso ayuda a entender por qué usamos índices para acceder a cada elemento.

Practica con ejemplos pequeños: Empieza con programas cortos (leer una edad, un nombre, 3 notas, 3 amigos) antes de intentar algo más grande. La idea es dominar bien Scanner y arreglos simples primero.

- **Código inicial utilizado en la exposición:**

```
// =====
// Archivo: HolaSalida.java
// =====
public class HolaSalida {
    public static void main(String[] args) {
        System.out.println("Hola, bienvenido al curso de programación en Java");
        System.out.println("Este es un ejemplo de salida por consola");
    }
}
// Fin de HolaSalida.java

// =====
// Archivo: LeerEdad.java
// =====
import java.util.Scanner;

public class LeerEdad {
    public static void main(String[] args) {
        // Creamos el Scanner para leer desde el teclado
        Scanner teclado = new Scanner(System.in);
```

```
        System.out.println("¿Cuántos años tienes?");
        int edad = teclado.nextInt(); // Leemos un número entero

        System.out.println("Tú tienes " + edad + " años.");

        teclado.close(); // Cerramos el Scanner
    }
}
// Fin de LeerEdad.java

// =====
// Archivo: LeerNombre.java
// =====
import java.util.Scanner;

public class LeerNombre {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.println("¿Cómo te llamas?");
        String nombre = teclado.nextLine(); // Leemos una línea de texto completa

        System.out.println("Hola " + nombre + ", ¡bienvenido a Java!");

        teclado.close();
    }
}
// Fin de LeerNombre.java

// =====
// Archivo: ArregloEnteros.java
// =====
public class ArregloEnteros {
    public static void main(String[] args) {
        // Arreglo de 3 notas
        int[] notas = new int[3];

        // Asignamos valores manualmente
        notas[0] = 15;
        notas[1] = 18;
        notas[2] = 20;

        System.out.println("Notas del estudiante:");
        System.out.println("Primera nota: " + notas[0]);
        System.out.println("Segunda nota: " + notas[1]);
        System.out.println("Tercera nota: " + notas[2]);
    }
}
// Fin de ArregloEnteros.java
```

```
// =====
// Archivo: ArregloCadenas.java
// =====
public class ArregloCadenas {
    public static void main(String[] args) {
        // Arreglo de 3 materias favoritas
        String[] materias = new String[3];

        materias[0] = "Matemática";
        materias[1] = "Comunicación";
        materias[2] = "Programación";

        System.out.println("Materias favoritas:");
        System.out.println("1: " + materias[0]);
        System.out.println("2: " + materias[1]);
        System.out.println("3: " + materias[2]);
    }
}
// Fin de ArregloCadenas.java


// =====
// Archivo: Main.java
// =====
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner teclado = new Scanner(System.in);

        String[] amigos = new String[3];

        System.out.println("Ingrese el nombre de tu primer amigo: ");
        amigos[0] = teclado.nextLine();

        System.out.println("Ingrese el nombre de tu segundo amigo: ");
        amigos[1] = teclado.nextLine();

        System.out.println("Ingrese el nombre de tu tercer amigo: ");
        amigos[2] = teclado.nextLine();

        System.out.println("Tus amigos son: ");
        System.out.println(amigos[0]);
        System.out.println(amigos[1]);
        System.out.println(amigos[2]);

        teclado.close();
    }
}
// Fin de Main.java
```

- **Práctica:** [Clic para programar](#)

Lección 4: Introducción a POO (clases, objetos, constructores), principios básicos: encapsulación, instanciación. (12 minutos)

Responsable: Schneider Carlos Alberto Delgado Carrasco

- **Descripción:** En esta lección se aplica la idea de clase como *molde* y objeto como *ejemplar real*. Creamos la clase **Student** con atributos, luego instanciamos objetos en **Main**. Agregamos un **constructor** para iniciar los datos en una sola línea y aplicamos **encapsulación** con **private**, getters y un setter con validación para proteger la edad.
- **Enlace del video:** https://www.youtube.com/watch?v=-_Zb4Olarh8
- **Editor online:** [Replit - POO Básico](#)
- **Consejos clave:**
 - Una clase es un molde; un objeto es un ejemplar de ese molde.
 - Podemos agrupar datos y comportamiento en una sola unidad.
 - Encapsulación ayuda a proteger los datos.
- **Código inicial utilizado en la exposición:**

Student.java

```
// CREACIÓN DE UNA CLASE
public class Student {
    String name;
    int age;

    // CREACIÓN DEL CONSTRUCTOR
    public Student(String name, int age){
        this.name = name;
        this.age = age;
    }

    // CREACIÓN DE ENCAPSULACIÓN

    // Getter
    public String getName(){
        return name;
    }

    public int getAge(){
        return age;
    }

    //Setter
```

```

    public void setAge(int age){
        if (age >= 0 && age <= 120){
            this.age= age;
        }
    }
}

```

Main.java

```

/** CREANDO OBJETOS
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Ana";
        s1.age = 15;

        Student s2 = new Student();
        s2.name = "Luis";
        s2.age = 16;

        System.out.println(s1.name + " tiene " + s1.age + " años.");
        System.out.println(s2.name + " tiene " + s2.age + " años.");
    }
}
*/

/** PARA EL CONSTRUCTOR
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Ana", 15);
        Student s2 = new Student("Luis", 16);

        System.out.println(s1.name + " tiene " + s1.age + " años.");
        System.out.println(s2.name + " tiene " + s2.age + " años.");
    }
}
*/

// PARA LA ENCAPSULACIÓN
public class Main {
    public static void main(String[] args) {

        Student s1 = new Student("Ana", 17);
        System.out.println(s1.getName() + " tiene " + s1.getAge() + " años. ");

        s1.setAge(19);
        System.out.println("Despues de actualizar, " + s1.getName() + " tiene " +
s1.getAge() + " años. ");

```

```
s1.setAge(25);
System.out.println("Edad final de " + s1.getName() + ": " + s1.getAge());

}

}
```

- **Práctica:** [Clic para programar](#)

Lección 5: Programa final POO, mejores practicas y errores comunes. (9 minutos)

Responsable: Alejandro Nicolas Barturen Guzman

- **Descripción:** En esta lección desarrollamos el proyecto final de la sección de Programación Orientada a Objetos: “Mi Alcantía Digital”, un programa sencillo que demuestra el uso de clases, objetos, atributos, métodos y encapsulación. Además, revisamos los errores más comunes que cometen los principiantes en Java —como comparar Strings con ==, exponer atributos públicos, y el uso de magic numbers— y mostramos la versión correcta siguiendo buenas prácticas, incluyendo código limpio, validaciones y nombres descriptivos.

La lección combina teoría, demostración en vivo y ejemplos comparativos de malas prácticas vs buenas prácticas, aplicados directamente al proyecto del Money Tracker.

- **Enlace del video:** [Mini Proyecto y Errores Comunes](#)
- **Editor online:** [JDoodle Proyecto Alcantia](#)
- **Consejos clave:**
 - Usa encapsulación: atributos private, acceso mediante métodos.
 - Usa nombres claros y descriptivos para métodos (evitar add() → mejor ahorrar() o gastar()).
 - Evita valores quemados (“magic numbers”) — usa constantes con final.
 - Valida todas las entradas del usuario.
 - No compares Strings con == → usar .equals().
 - Usa constructores para iniciar valores por defecto.
 - Mantén el código corto, ordenado y comentado solo cuando sea necesario.
- **Código inicial utilizado en la exposición:**

```
-
public class Alcantia {
```

```
private double saldo;

public Alcantia() {
    this.saldo = 0.0;
}

public double verSaldo() {
    return this.saldo;
}

public boolean ahorrar(double cantidad) {
    if (cantidad <= 0) {
        return false;
    }
    this.saldo += cantidad;
    return true;
}

public boolean gastar(double cantidad) {
    if (cantidad <= 0 || cantidad > this.saldo) {
        return false;
    }
    this.saldo -= cantidad;
    return true;
}

public static void main(String[] args) {
    Alcantia miAlcantia = new Alcantia();
    System.out.println("Saldo inicial: " + miAlcantia.verSaldo());

    miAlcantia.ahorrar(10.0);
    System.out.println("Después de ahorrar 10: " + miAlcantia.verSaldo());

    miAlcantia.gastar(3.5);
    System.out.println("Después de gastar 3.5: " + miAlcantia.verSaldo());
}
}
```

- **Práctica:** [Clic para programar](#)

Recursos Adicionales

- **Código Fuente Completo:** [Repositorio de GitHub](#)
- **Todas las Actividades Prácticas:**

Recursos Adicionales

Lección	Actividad	Editor Online / Enlace
1	Introducción a Java y la programación, sintaxis básica: variables, tipos de datos.	https://replit.com/join/ewucjjgzne-andreows
2	Operadores y estructuras de control, métodos, funciones y parámetros.	https://replit.com/@matadorxdm/Java
3	Entrada/salida con Scanner, Arreglos simples o cadenas.	https://replit.com/@smbmontalvo/JavaCourse
4	Introducción a la POO (clases, objetos, constructores), principios básicos: encapsulación, instanciación.	https://replit.com/join/jzpobhqipb-schneider0104dc
5	Programa final POO, mejores practicas y errores comunes	https://www.jdoodle.com/ga/tlNinqaBOJ0slkDDMK2rAQ%3D%3D

¡Gracias por completar el curso!

Elaboración

Universidad Peruana de Ciencias Aplicadas Carrera de Ingeniería de Software Período 202520 Curso: 1ASI0729 Desarrollo de Aplicaciones Open Source NRC 7391 **Nombre del equipo:** chronisteam **Líder del equipo:** Alejandro Nicolas Barturen Guzman **Integrantes del equipo:** Andreow Jomark Santiago Peña, Carlos Alberto Lopez Goitia, , Schneider Carlos Alberto Delgado Carrasco & Sebastian Martin Beingolea Montalvo **Fecha de entrega:** 16-11-2025

Anexos

Enlaces de videos de las lecciones

1. Lección 1: [Introducción a Java y la programación, sintaxis básica](#)
2. Lección 2: [Operadores y estructuras de control, métodos, funciones y parámetros](#)
3. Lección 3: [Entrada/salida con Scanner, Arreglos simples o cadenas](#)
4. Lección 4: [Introducción a POO \(clases, objetos, constructores\), principios básicos](#)
5. Lección 5: [Programa final POO, mejores prácticas y errores comunes](#)

Enlaces de códigos fuente utilizados

1. Lección 1: [Replit - HolaMundo.java](#)
2. Lección 2: [Replit - Operadores y estructuras de control, métodos, funciones y parámetros](#)
3. Lección 3: [Replit - Entrada/salida con Scanner, Arreglos simples o cadenas](#)

4. Lección 4: [Replit - POO Básico](#)
5. Lección 5: [JDoodle - Proyecto Alcancía](#)